

UNIVERSIDAD DEL VALLE DE GUATEMALA
Construcción de Compiladores
Sección 20



José Gerardo Ruiz García - 23719
Gerardo André Fernández Cruz - 23763
Juan Diego Solís Martínez - 23720
Jose Auyón - 201579

GUATEMALA, 6 de septiembre 2026

Arquitectura del Compilador	3
1. Propósito de esta parte	3
2. Pipeline de compilación	3
3. La gramática (compiler/Compiscript.g4)	4
4. El sistema de tipos (compiler/semantic/types.py)	4
4.1 Reglas de compatibilidad de tipos	5
5. Tabla de símbolos y manejo de ámbitos (compiler/semantic/symbol_table.py)	5
5.1 Symbol	5
5.2 Scope y el modelo de ámbitos como árbol	5
5.3 SymbolTable: la pila de ámbitos activos	6
5.4 Redefinición vs. shadowing	6
6. El analizador semántico (compiler/semantic/semantic_analyzer.py)	6
6.1 El patrón Visitor y la propagación de tipos	7
6.2 Estrategia de acumulación de errores (no fail-fast)	7
6.3 Dos pasadas para referencias hacia adelante entre clases	7
6.4 Resolución de cadenas de acceso (obj.metodo())[0].campo)	7
6.5 Detección de código muerto	8
6.6 Mapa de reglas semánticas a implementación	8
7. Manejo de errores (compiler/semantic/errors.py)	9
8. Visualizador de AST (compiler/ast_visualizer.py)	9
9. El driver (compiler/driver.py)	9
10. IDE web (ide/)	9
11. Estructura de carpetas del repositorio	10
12. Decisiones de diseño relevantes	10
Guía de Ejecución	11
1. Requisitos previos	11
2. Instalación paso a paso	11
2.1 Linux / macOS	11
2.2 Windows (PowerShell)	11
2.3 Con Docker	12
3. Analizar un archivo desde la línea de comandos	12
4. Correr pruebas	13
5. Usar el IDE web	14
6. Estructura mínima de un archivo .cps	14
Enlaces	14
1. Repositorio de GitHub	14
1. Enlace al video de YouTube	14

Arquitectura del Compilador

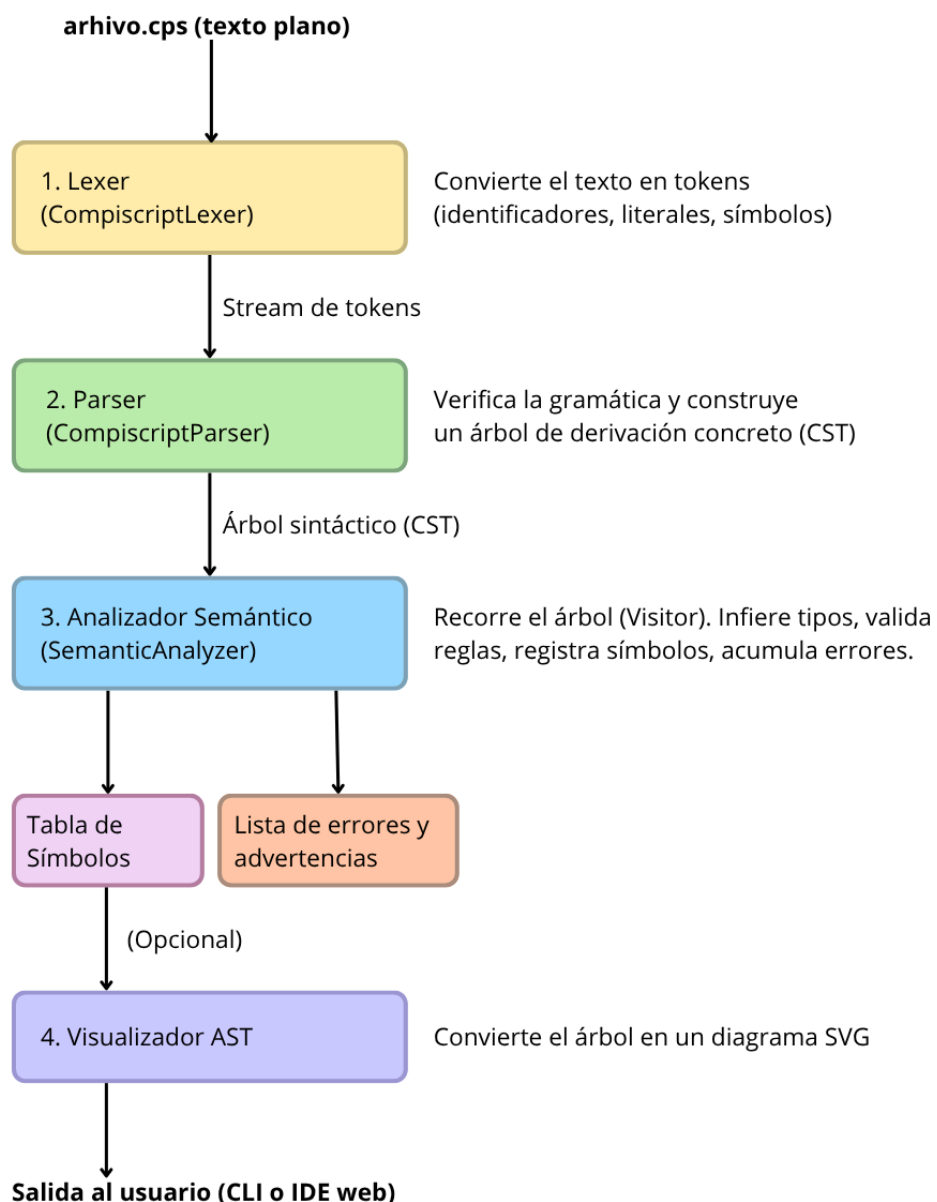
1. Propósito de esta parte

Esta sección describe cómo está construido internamente el compilador de Compiscript: qué componentes existen, qué responsabilidad tiene cada uno, cómo se comunican entre sí, y por qué se tomaron ciertas decisiones de diseño. Está pensada tanto para quien deba mantener o extender el código, como para quien deba evaluarlo.

2. Pipeline de compilación

Compiscript, en esta fase, no genera código ejecutable ni interpreta el programa: su trabajo termina en la verificación semántica. El resultado final de compilar un archivo .cps es una lista de errores/advertencias, una tabla de símbolos completa, y (opcionalmente) una representación visual del árbol sintáctico.

El flujo de una compilación es el siguiente:



Las etapas 1 y 2 son generadas automáticamente por ANTLR4 a partir de la gramática (Compiscript.g4); el equipo no escribe ese código a mano. Las etapas 3 y 4, junto con la tabla de símbolos y el propio driver que orquesta todo, sí son implementación propia.

3. La gramática (compiler/Compiscript.g4)

La gramática define la sintaxis completa del lenguaje: declaraciones, sentencias de control de flujo, clases, y un sistema de expresiones con precedencia de operadores explícita (de menor a mayor precedencia: asignación → ternario → || → && → igualdad → relacional → aditiva → multiplicativa → unaria → primaria).

Dos decisiones de la gramática vale la pena resaltar porque afectan directamente al analizador semántico:

- leftHandSide como cadena de sufijos

Cualquier expresión que empiece con un identificador, this, o new puede seguir con cualquier combinación de (...) (llamada), [...] (indexación) y .identificador (acceso a propiedad), en cualquier orden y cantidad (por ejemplo obj.metodo()[0].campo). Esto simplifica mucho el análisis: en vez de tener una regla distinta por cada combinación, el analizador semántico recorre esta cadena de sufijos una sola vez, aplicando cada uno sobre el tipo resultante del anterior.

- Tipo float agregado

La gramática original del curso no incluía float como tipo base ni como literal. El equipo lo añadió porque el requerimiento de "verificación de tipos en operaciones aritméticas" exige explícitamente aceptar tanto integer como float, y sin este tipo no habría manera de expresar ni probar esa regla.

4. El sistema de tipos (compiler/semantic/types.py)

Todo tipo en Compiscript es una instancia de una jerarquía de clases con una raíz común, CompiscriptType, que define dos operaciones compartidas por todos los tipos:

Método	Propósito
is_numeric()	True solo para integer y float. Se usa en toda validación aritmética/relacional.
is_compatible_with(otro)	Determina si un valor de este tipo puede usarse donde se espera 'otro' (asignaciones, retornos, argumentos, comparaciones).

Las clases concretas son:

- PrimitiveType: envuelve los tipos atómicos (integer, float, string, boolean, null, void, any, error). Son singletons, hay una única instancia de INTEGER, una de BOOLEAN, etc., reutilizada en todo el programa.

- **ArrayType:** guarda el tipo de elemento y el número de dimensiones (`integer[][]` se representa como `ArrayType(INTEGER, dimensions=2)`), no como un array de arrays anidado. Esto simplifica comparar e indexar arreglos multidimensionales.
- **FunctionType:** guarda la lista de tipos de parámetros, el tipo de retorno y (para fines de mensajes de error) los nombres de los parámetros.
- **ClassType:** guarda el nombre de la clase, un puntero opcional a su superclase, y dos diccionarios (`attributes`, `methods`) que mapean `nombre` → `tipo`. `resolve_member(nombre)` busca un atributo o método primero en la clase misma y, si no lo encuentra, sube recursivamente por la cadena de herencia.

4.1 Reglas de compatibilidad de tipos

- `is_compatible_with` no es una simple igualdad. Las reglas, en orden de evaluación:
- Si cualquiera de los dos tipos es `any` o `error`, siempre son compatibles. Esto evita las llamadas cascadas de errores (un solo error real disparando docenas de errores secundarios sin sentido).
- Un `integer` es compatible con un `float` (pero no al revés), la única coerción implícita del lenguaje, igual que en `TypeScript/JavaScript`.
- Para clases, un objeto de tipo `Subclase` es compatible con el tipo `Superclase` (y cualquier ancestro), pero no al revés, permite `let a: Animal = new Perro(...)`.
- Para arreglos, son compatibles si sus tipos de elemento son compatibles entre sí y tienen la misma cantidad de dimensiones.
- En cualquier otro caso, se requiere igualdad estricta de tipo.

5. Tabla de símbolos y manejo de ámbitos (`compiler/semantic/symbol_table.py`)

5.1 Symbol

Cada identificador declarado (variable, constante, parámetro, función, clase, o variable de `foreach`) se representa como un objeto `Symbol` con: `nombre`, `kind` (`VARIABLE/CONSTANT/FUNCTION/PARAMETER/CLASS/LOOP_VAR`), `tipo de dato`, `línea` y `columna de declaración`, `nivel de ámbito`, si ya fue inicializado, y si es constante.

5.2 Scope y el modelo de ámbitos como árbol

Un `Scope` es un entorno: un diccionario `nombre` → `Symbol` más un puntero a su `Scope` padre. Cada `Scope` tiene además una etiqueta (`scope_kind`) que indica qué tipo de bloque lo originó: `global`, `function`, `class`, `block`, `loop`, o `switch`. Esta etiqueta es clave porque varias reglas semánticas dependen de saber en qué tipo de contexto está el analizador en un punto dado del recorrido, no solo de qué variables son visibles.

La búsqueda de un identificador (`lookup`) empieza en el `Scope` actual y, si no lo encuentra, sube recursivamente por la cadena de padres hasta llegar al ámbito global. Esta única función implementa a la vez:

- La resolución de nombres local vs. global que pide el requerimiento.

- El soporte de closures: una función anidada definida dentro de otra puede "ver" las variables de la función contenedora simplemente porque su Scope tiene como padre el Scope de la función exterior, no hace falta ningún mecanismo especial de captura.

5.3 SymbolTable: la pila de ámbitos activos

SymbolTable mantiene una pila (`_scope_stack`) de los ámbitos abiertos en el punto actual del recorrido. `enter_scope(kind=...)` empuja un nuevo Scope y `exit_scope()` lo saca. El analizador semántico llama a estas dos funciones al entrar y salir de cada bloque `{...}`, cuerpo de función, cuerpo de clase, cuerpo de loop, y rama de switch: cada construcción que introduce un nuevo ámbito léxico en la gramática tiene su propio par `enter_scope/exit_scope`.

Sobre esta pila se construyen varias funciones de contexto que el analizador consulta constantemente:

Función	Responde a la pregunta...
<code>is_inside_loop()</code>	¿Hay un ámbito 'loop' entre aquí y la función o el global más cercano? (para <code>continue</code>)
<code>is_break_allowed()</code>	Igual, pero también acepta 'switch' (para <code>break</code>)
<code>is_inside_function()</code>	¿Hay algún ámbito 'function' en la pila? (para <code>return</code>)
<code>is_inside_class()</code>	¿Hay algún ámbito 'class' en la pila? (para <code>this</code>)
<code>current_class_type()</code>	¿Cuál es el <code>ClassType</code> de la clase que envuelve el punto actual?

Estas funciones recorren la pila de arriba hacia abajo y se detienen al toparse con un `function` o `global`: esto hace que un `continue` dentro de un `while` anidado dentro de una función funcione, pero que ese mismo `continue`, si apareciera en una función anidada dentro de ese `while`, sea correctamente rechazado, el límite de una función corta la visibilidad de estar dentro de un loop.

5.4 Redeclaración vs. shadowing

`Scope.define(symbol)` devuelve `False` si el nombre ya existe en ese mismo Scope (no en un padre). Esto tiene una consecuencia importante: declarar una variable `x` dentro de un bloque cuando ya existe una `x` en el ámbito exterior no es un error, es shadowing, válido en la mayoría de lenguajes con ámbito léxico. Solo se rechaza declarar el mismo nombre dos veces dentro del mismo bloque.

6. El analizador semántico (compiler/semantic/semantic_analyzer.py)

Es la pieza central del proyecto: una clase `SemanticAnalyzer` que extiende `CompiscriptVisitor` (generada automáticamente por ANTLR). Por cada regla de la gramática que necesita validación existe un método `visitNombreDeLaRegla`.

6.1 El patrón Visitor y la propagación de tipos

Cada método `visit*` de una expresión retorna el `CompiscriptType` inferido de esa expresión. Esto permite que el análisis de tipos se propague de abajo hacia arriba: `visitAdditiveExpr` llama a `self.visit()` sobre sus operandos, obtiene sus tipos, decide si la operación es válida, y retorna el tipo resultante. Los métodos `visit*` de sentencias no retornan un tipo útil, su trabajo es validar y modificar la tabla de símbolos como efecto secundario.

6.2 Estrategia de acumulación de errores (no fail-fast)

El analizador nunca se detiene en el primer error semántico. Todos los errores se acumulan en un `ErrorCollector` y se reportan juntos al final, igual que un compilador de producción.

El mecanismo que lo hace viable sin generar cascadas sin sentido es el tipo especial `ERROR_TYPE`: cuando una expresión no puede tipificarse correctamente, el analizador reporta un error y devuelve `ERROR_TYPE` en vez de detenerse. Como `ERROR_TYPE` es compatible con cualquier otro tipo, ninguna validación posterior que lo involucre vuelve a fallar por la misma causa.

6.3 Dos pasadas para referencias hacia adelante entre clases

`visitProgram` no visita el árbol de una sola vez. Primero hace un barrido rápido (`_pre_register_class`) que registra el nombre de cada clase declarada a nivel global, creando un `ClassType` vacío para cada una, antes de analizar el cuerpo de ninguna. Recién después hace la visita completa y normal del árbol.

Esto es necesario porque una clase puede heredar de otra declarada más abajo en el archivo, o un método puede recibir como parámetro un tipo de una clase declarada después. El mismo patrón se repite, a menor escala, dentro de `visitClassDeclaration`: primero se registran todos los atributos y métodos de la clase, y luego se visita el cuerpo de cada miembro.

6.4 Resolución de cadenas de acceso (`obj.metodo()[0].campo`)

El método `visitLeftHandSide` visita el átomo inicial para obtener su tipo, y luego itera sobre cada sufijo llamando a `_apply_suffix`, que según el tipo de sufijo:

- Llamada (...): exige que el tipo actual sea `FunctionType`, valida cantidad y tipo de cada argumento posicionalmente, y el tipo resultante pasa a ser el `return_type` de la función.
- Indexación [...]: exige que el tipo actual sea `ArrayType`, valida que el índice sea `integer`, y el tipo resultante es el tipo de elemento (o un `ArrayType` de una dimensión menos).
- Acceso .propiedad: exige que el tipo actual sea `ClassType`, resuelve el miembro (incluyendo herencia), y el tipo resultante es el tipo de ese miembro.

Esto permite validar semánticamente expresiones arbitrariamente encadenadas sin necesidad de una regla especial por cada combinación posible.

6.5 Detección de código muerto

`_visit_block_dead_code` reemplaza la visita normal de los enunciados de un bloque: recorre las sentencias en orden y, apenas encuentra un `return`, `break` o `continue`, marca todo lo que venga después como inalcanzable y reporta un error por cada sentencia sobrante, sin dejar de visitarlas, para seguir detectando otros errores dentro de código muerto.

6.6 Mapa de reglas semánticas a implementación

Categoría del requerimiento	Método(s) responsable(s)
Tipos en operaciones aritméticas	<code>visitAdditiveExpr</code> , <code>visitMultiplicativeExpr</code>
Tipos en operaciones lógicas	<code>visitLogicalOrExpr</code> , <code>visitLogicalAndExpr</code> , <code>visitUnaryExpr</code>
Compatibilidad en comparaciones	<code>visitEqualityExpr</code> , <code>visitRelationalExpr</code>
Tipos en asignaciones	<code>visitVariableDeclaration</code> , <code>visitAssignment</code> , <code>visitAssignExpr</code>
Inicialización obligatoria de <code>const</code>	Gramática ('=' obligatorio) + <code>visitConstantDeclaration</code>
Tipos en arreglos e índices	<code>visitArrayLiteral</code> , <code>_apply_suffix</code> (<code>IndexExprContext</code>)
Resolución de nombres / no declaradas	<code>visitIdentifierExpr</code> , <code>SymbolTable.lookup</code>
Redeclaración en el mismo ámbito	<code>Scope.define</code> (valor de retorno)
Nuevos entornos por función/clase/bloque	<code>enter_scope</code> / <code>exit_scope</code> en cada <code>visit*</code> relevante
Argumentos de llamadas (número y tipo)	<code>_apply_suffix</code> (<code>CallExprContext</code>), <code>visitNewExpr</code>
Tipo de retorno de función	<code>visitReturnStatement</code>
Recursión	Definición del símbolo antes de visitar su propio cuerpo
Funciones anidadas / closures	Cadena de ámbitos (sección 5.2)
Doble declaración de función	<code>visitFunctionDeclaration</code> (chequeo con <code>lookup_local</code>)
Condición booleana en <code>if/while/do-while/for</code>	<code>_check_boolean</code>
<code>break/continue</code> solo en bucles	<code>visitBreakStatement</code> , <code>visitContinueStatement</code>
<code>return</code> solo dentro de función	<code>visitReturnStatement</code>
Atributos/métodos válidos vía <code>.</code>	<code>ClassType.resolve_member</code>
Verificación de llamada al constructor	<code>visitNewExpr</code>

this con verificación de ámbito	visitThisExpr
Código muerto	_visit_block_dead_code
Expresiones sin sentido (multiplicar funciones)	FunctionType/ClassType no son is_numeric()
Declaraciones/parámetros duplicados	visitVariableDeclaration, chequeo en visitFunctionDeclaration

7. Manejo de errores (compiler/semantic/errors.py)

Es deliberadamente minimalista: una clase `SemanticError` (mensaje, línea, columna, severidad) y un `ErrorCollector` que los acumula, exponiendo filtros (`errors_only()`, `warnings_only()`) y un chequeo (`has_errors()`). La distinción entre `ERROR` y `WARNING` permite que situaciones ambiguas o de bajo riesgo se reporten sin impedir que el programa se considere válido, el driver solo devuelve código de salida distinto de cero si hay al menos un `ERROR`.

8. Visualizador de AST (compiler/ast_visualizer.py)

Recorre el mismo árbol de parseo (CST) que consume el analizador semántico, pero en vez de validar nada, construye un grafo con `graphviz`: cada nodo interno de la gramática se dibuja como un nodo, cada token terminal como una hoja, y las aristas representan la relación padre-hijo del árbol. El color de cada nodo varía según su categoría para hacer el árbol más legible. El resultado se renderiza a SVG, tanto para guardarse como archivo (CLI, con `--ast-out`) como para incrustarse directamente en el navegador (IDE web).

9. El driver (compiler/driver.py)

Es el punto de entrada único que orquesta las cuatro etapas del pipeline. Su función central, `compile_source`, recibe el texto fuente y devuelve un diccionario con `errors`, `warnings`, `symbol_table` y, opcionalmente, `ast/ast_file`. Esta función es reutilizada tanto por la interfaz de línea de comandos como por el backend del IDE web y por el harness de tests: hay un único lugar que sabe cómo compilar un programa.

Un detalle de robustez: si el `lexer/parser` produce errores sintácticos, `compile_source` corta el proceso ahí mismo y no intenta correr el análisis semántico sobre un árbol posiblemente incompleto o mal formado.

10. IDE web (ide/)

- Backend (`ide/server.py`): un servidor Flask con dos rutas relevantes. `GET /` sirve el frontend estático. `POST /compile` recibe `{ code, ast }` y devuelve `{ errors, warnings, symbol_table, ast_svg }` como JSON. Cualquier excepción no controlada se atrapa y se devuelve como error de fase 'internal' en vez de tumbar el servidor.

- Frontend (ide/static/): un editor de texto con numeración de líneas, panel de diagnósticos por línea y columna, árbol de símbolos navegable, y un panel con el SVG del AST. Todo en HTML/CSS/JS plano, sin build step ni framework.

11. Estructura de carpetas del repositorio

```

compiscript-compiler/
├── compiler/
│   ├── Compiscript.g4      # Gramática ANTLR4
│   ├── driver.py           # Orquestador del pipeline
│   ├── ast_visualizer.py   # Render del AST a SVG
│   ├── generated/         # (No versionado) Lexer/Parser generados
│   └── semantic/
│       ├── types.py        # Sistema de tipos
│       ├── symbol_table.py # Symbol, Scope, SymbolTable
│       ├── errors.py       # SemanticError, ErrorCollector
│       └── semantic_analyzer.py # El Visitor con las reglas semánticas
├── ide/
│   ├── server.py           # Backend Flask
│   └── static/             # Frontend (HTML/CSS/JS)
├── program/
│   ├── Driver.py           # Entry point mínimo exigido por el curso
│   └── program.cps         # Archivo de prueba de ejemplo
├── tests/
│   ├── success/            # Programas que deben compilar sin error
│   ├── failure/            # Programas con errores intencionales
│   └── run_tests.py        # Harness de pruebas
├── docs/                   # Documentación del proyecto
├── antlr-4.13.2-complete.jar # Herramienta de generación
├── setup.sh                # Script de instalación (Linux/macOS)
├── Dockerfile
└── requirements.txt

```

12. Decisiones de diseño relevantes

- Inferencia de tipos en declaraciones sin anotación (`let x = 5;`): el tipo final se toma del inicializador, reduciendo verbosidad sin renunciar a la verificación de tipos posterior.
- integer es assignable a float, no al revés: modela la coerción numérica estándar de widening, evitando falsos errores en código aritmético normal.
- Acumulación de errores con tipo centinela (`ERROR_TYPE`) en vez de abortar en el primer error: refleja el comportamiento esperado de un compilador real y evita cascadas de errores redundantes.
- Registro de clases en dos pasadas: necesario para soportar referencias hacia adelante entre clases y entre miembros de una misma clase.
- Resolución de ámbitos vía cadena de padres en memoria: hace que el soporte de closures 'salga gratis' de la implementación general de ámbitos, sin un mecanismo de captura aparte.

Guía de Ejecución

Este apartado explica, paso a paso, cómo dejar el proyecto funcionando desde cero, cómo usar cada una de sus herramientas (analizador por consola, batería de pruebas, IDE web), y cómo resolver los problemas más comunes que pueden aparecer al instalarlo.

1. Requisitos previos

Herramienta	Uso	Cómo verificar
Java 11+	Ejecutar la herramienta ANTLR4 (.jar), que traduce la gramática a código Python	java -version
Python 3.10+	Ejecutar todo el compilador (driver, tests, IDE)	python3 --version
pip	Instalar las dependencias de Python	pip --version
Graphviz (binario)	Renderizar el AST como imagen SVG	dot -V

2. Instalación paso a paso

2.1 Linux / macOS

El proyecto incluye setup.sh, un script de Bash que automatiza todo lo de esta sección.

```
cd compiscript-compiler
chmod +x setup.sh
./setup.sh
```

Este script: descarga el .jar de ANTLR si no está presente, verifica que Java esté instalado, instala las dependencias de Python (antlr4-python3-runtime, flask, flask-cors, graphviz), genera el lexer/parser a partir de la gramática, y hace una verificación final importando el parser generado.

2.2 Windows (PowerShell)

Importante: setup.sh es un script de Bash y no se ejecuta de forma nativa en PowerShell ni en cmd.exe. En Windows sin WSL, hay que hacer estos mismos pasos manualmente.

```
cd compiscript-compiler

# 1. Instalar dependencias de Python
pip install antlr4-python3-runtime==4.13.2 flask flask-cors graphviz

# 2. Crear la carpeta donde ANTLR va a dejar el parser generado
mkdir compiler\generated
```

```
# 3. Generar el lexer y el parser a partir de la gramática
java -jar antlr-4.13.2-complete.jar -Dlanguage=Python3 -visitor -listener -o compiler\generated
compiler\Compiscript.g4

# 4. Crear el __init__.py que Python necesita
New-Item -ItemType File -Path compiler\generated\__init__.py

# 5. Verificar que el parser se generó correctamente
dir compiler\generated
```

El último comando debería mostrar archivos como `CompiscriptLexer.py`, `CompiscriptParser.py`, `CompiscriptVisitor.py` y `CompiscriptListener.py` (posiblemente dentro de una subcarpeta `compiler\generated\compiler\`, dependiendo de la versión de ANTLR, el código del proyecto ya contempla ambas ubicaciones automáticamente).

Alternativa recomendada si el equipo usa mezcla de sistemas operativos: instalar Git Bash (incluido con Git para Windows) o usar WSL, y correr `setup.sh` normalmente desde ahí, evita mantener dos procedimientos de instalación distintos.

2.3 Con Docker

```
# Construir la imagen (esto ya genera el parser dentro de la imagen)
docker build --rm . -t csp-image

# Levantar un contenedor con el IDE expuesto en el puerto 8080
docker run --rm -ti -p 8080:5000 -v "$(pwd)/program":/program csp-image
```

Con esto el IDE queda disponible en `http://localhost:8080` sin necesidad de instalar Java, Python ni Graphviz en la máquina anfitriona. Es la vía más confiable si se quiere evitar cualquier diferencia entre sistemas operativos del equipo.

3. Analizar un archivo desde la línea de comandos

El punto de entrada principal es `compiler/driver.py`:

```
python3 compiler/driver.py program/program.cps
```

Si el archivo no tiene errores, la salida es simplemente:

```
✓ No errors found in 'program/program.cps'
```

Si hay errores o advertencias, se listan ordenados por línea y columna, indicando la fase en la que se detectaron (syntax o semantic):

```
[ERROR] [semantic] Line 4:8 , Type mismatch in declaration of 'x':
declared 'integer', got 'string'
1 error(s), 0 warning(s)
```

Opciones disponibles

Flag	Efecto
--symbols	Imprime la tabla de símbolos completa, en JSON, con todos los ámbitos anidados
--ast	Imprime el árbol sintáctico como JSON
--ast-out RUTA	Renderiza el árbol sintáctico como SVG y lo guarda en RUTA.svg (requiere Graphviz)
--json	Toda la salida en un único bloque JSON, útil para integrarlo con otras herramientas

Ejemplos:

```
python3 compiler/driver.py program/program.cps --symbols --ast
python3 compiler/driver.py program/program.cps --ast-out output/arbol
python3 compiler/driver.py program/program.cps --json
```

Código de salida: el proceso termina con código 0 si no hubo errores (aunque haya advertencias), y con código 1 si hubo al menos un error. Esto permite usarlo directamente en scripts de CI/CD sin tener que parsear la salida por texto.

4. Correr pruebas

```
python3 tests/run_tests.py
```

El runner recorre automáticamente todos los archivos .cps dentro de tests/success/ (que deben compilar sin ningún error) y tests/failure/ (que deben producir al menos un error), y reporta cuántos pasaron.

Opciones útiles:

```
python3 tests/run_tests.py --verbose      # detalle de cada error/warning
python3 tests/run_tests.py --filter classes # solo tests cuyo nombre contiene "classes"
python3 tests/run_tests.py --json         # resumen en JSON al final
```

5. Usar el IDE web

```
python3 ide/server.py
```

Por defecto arranca en `http://127.0.0.1:5000`. Ábrelo en el navegador.

Desde el IDE se puede:

- Escribir o pegar código Compiscript en el editor (con numeración de línea y soporte de Tab).
- Compilarlo con el botón ► o con Ctrl+Enter.
- Ver los errores y advertencias en un panel de diagnósticos, cada uno con su línea y columna.
- Explorar la tabla de símbolos como un árbol navegable de ámbitos.
- Ver el árbol sintáctico (AST) renderizado visualmente (requiere Graphviz instalado).
- Cargar ejemplos de código predefinidos para probar rápidamente distintas construcciones del lenguaje.

Si se va exponer fuera de localhost (por ejemplo, dentro de Docker), el servidor puede configurarse con variables de entorno:

```
HOST=0.0.0.0 PORT=5000 python3 ide/server.py
```

6. Estructura mínima de un archivo .cps

Todo archivo fuente de Compiscript debe usar la extensión `.cps`. No hay una sección obligatoria de "main": el archivo es simplemente una secuencia de sentencias que se analizan en orden de arriba hacia abajo:

```
let contador: integer = 0;

function incrementar(): integer {
  contador = contador + 1;
  return contador;
}

print(incrementar());
```

Enlaces

1. Repositorio de GitHub

<https://github.com/jruiz002/compiscript-compiler.git>

1. Enlace al video de YouTube

<https://youtu.be/dK3sldFkwLs>